

Personal Project Plan

AI-Powered Weekly Schedule Planner

1. Project Overview

This project is an AI-powered schedule planning application that helps users manage internships, part-time jobs, studying, personal tasks, social plans, and rest time in a more balanced way.

The application connects with Google Calendar to understand the user's existing schedule, then uses AI to suggest a realistic weekly plan based on the user's priorities, availability, workload, and personal preferences.

Rather than simply finding empty time slots, the app aims to recommend schedules that are sustainable and realistic. It helps users avoid overbooking themselves while still making progress on important goals.

2. Problem Statement

Many students and young professionals use calendar apps to record their schedules, but they still struggle with deciding how much they can realistically take on.

For example, a user may technically be free on Thursday afternoon, but that does not mean they should accept a work shift during that time. They may have an interview the next day, a major assignment due soon, or simply need time to rest.

Existing calendar apps mainly show availability. They do not fully consider personal capacity, stress levels, weekly priorities, or the mental load of different commitments.

This project addresses that gap by helping users make better scheduling decisions based on both their calendar and their personal context.

3. Target Users

The main target users are:

University students

Students who are balancing classes, assignments, internships, part-time jobs, clubs, social life, and personal responsibilities.

Internship and part-time job seekers

Users who need to submit shift availability or plan working hours around interviews, study time, and other commitments.

People who tend to overbook themselves

Users who often say yes to too many commitments because their calendar looks technically free, even when they are already mentally or physically overloaded.

International students

Students who may need to manage different time zones, remote internships, temporary stays in different countries, and changing weekly schedules.

4. Core Goal

The goal of this application is to help users answer questions such as:

When should I submit my shift availability this week?

How many work shifts can I realistically take?

Which days should I keep free for studying or rest?

Am I overbooking myself?

Where can I fit internship work, personal study, and social plans without burning out?

The app should act as a personal scheduling assistant that supports better decision-making, rather than simply displaying calendar events.

5. Minimum Viable Product

The first version should focus on a simple but useful workflow.

5.1 Google Calendar Integration

The app connects to the user's Google Calendar and retrieves upcoming events for the selected week.

The app should collect basic event information such as:

- Event title
- Date
- Start time
- End time
- Duration
- Calendar category

The app uses this information to identify busy periods and available time blocks.

5.2 User Profile Settings

The user can enter personal scheduling preferences.

Example settings:

- Minimum sleep: 7 hours per night
- Maximum part-time job shifts per week: 2
- Maximum internship hours per week: 20
- Preferred work time: morning or afternoon
- Avoid scheduling too many events in one day
- Avoid working the day before an interview
- Leave at least one evening free per week

These preferences should be stored and reused each week.

5.3 Weekly Priority Input

Each week, the user can enter their current priorities.

Example:

This week, I want to prioritize internship interviews and self-study.
I can work part-time up to two days.

I also want to keep one day lighter because I have been feeling tired.

This allows the AI to adjust its recommendations based on the user's current situation.

5.4 AI Schedule Recommendation

The app sends the calendar events, user preferences, and weekly priorities to the AI model.

The AI then returns a structured weekly recommendation.

Example output:

```
{
  "weekly_summary": "This week is relatively busy, so the user should prioritize internship preparation and limit part-time work to two shifts.",
  "recommended_shift_times": [
    {
      "day": "Monday",
      "time": "10:00-15:00",
      "reason": "There are no major events before or after this time, and the user still has enough time for studying in the evening."
    },
    {
      "day": "Friday",
      "time": "09:00-13:00",
      "reason": "This leaves the afternoon open for rest and personal tasks."
    }
  ],
  "avoid_times": [
    {
      "day": "Wednesday",
      "reason": "The user has an internship interview and should avoid adding extra commitments."
    }
  ],
  "warnings": [
    "The user should avoid adding more than two work shifts this week."
  ]
}
```

5.5 Shift Availability Generator

The app creates a clean shift availability list that the user can submit to an internship or part-time job.

Example:

Available times this week:

Monday: 10:00 AM - 3:00 PM

Wednesday: Not available

Friday: 9:00 AM - 1:00 PM

Sunday: 12:00 PM - 5:00 PM

Preferred maximum: 2 shifts this week

This makes the app immediately useful, even in its first version.

6. Main Features

6.1 Weekly Dashboard

The dashboard shows:

Upcoming events

Suggested work slots

Suggested study blocks

Recommended rest time

Weekly workload warning

The user should be able to quickly understand whether their week is balanced or overloaded.

6.2 Capacity Score

The app can calculate a simple “weekly capacity score” based on how busy the user is.

Example:

Low load: 0-40%

Moderate load: 41-70%

High load: 71-90%

Overloaded: 91-100%

This does not need to be perfect at first. It can be based on simple rules such as number of scheduled hours, number of work shifts, number of high-stress events, and amount of free time.

6.3 Event Load Levels

Each event can be assigned a load level.

Example:

Lunch with a friend: Low load

Part-time job shift: Medium load

Internship interview: High load

Exam or project deadline: Very high load

This helps the AI understand that not all events affect the user equally.

6.4 AI Explanation

The AI should explain why it recommends certain time slots.

Example:

Friday morning is recommended because it is a free time block, and the user has no major events the previous evening. This makes it a safer option for a part-time job shift.

This makes the app feel more trustworthy and useful.

7. Future Features

7.1 Mood and Energy Tracking

The user can log their mood, stress, and energy level.

Example:

Energy level: Medium

Stress level: High

Mood: Anxious

Notes: I have a lot of interviews this week.

The AI can use this information to suggest a lighter schedule when needed.

7.2 Calendar Event Creation

After reviewing the AI-generated schedule, the user can add selected events to Google Calendar.

For safety, the AI should not automatically edit the calendar without user confirmation.

7.3 Weekly Reflection

At the end of the week, the app asks the user to reflect.

Example questions:

Was this week too busy?

Did you get enough rest?

Were the shift recommendations realistic?

What should the app change for next week?

This feedback can improve future recommendations.

7.4 Reusable Scheduling Rules

The user can create custom rules.

Examples:

Do not schedule work after 8 PM.

Do not schedule more than two high-load events in one day.

Keep Sunday evening free.

Leave at least 2 hours before and after interviews.

Do not schedule part-time work on days with major deadlines.

8. Technical Stack

Frontend

Next.js
React
Tailwind CSS

Next.js would be a good choice because it works well for full-stack web apps and is useful for portfolio projects.

Backend

Next.js API Routes
Node.js

The backend handles API requests, authentication, and communication with Google Calendar and OpenAI.

Database

Supabase

Supabase can store:

User preferences
Weekly priorities
Mood and energy logs
AI-generated recommendations
Saved schedules
Custom rules

External APIs

Google Calendar API
OpenAI API

Google Calendar API is used to read calendar events and optionally create new events.

OpenAI API is used to generate schedule recommendations, explain decisions, and convert user preferences into structured planning rules.

9. Data Flow

The basic data flow could look like this:

User logs in

↓

App fetches Google Calendar events

↓

User enters weekly priorities

↓

App combines calendar data + preferences + priorities

↓

Data is sent to the AI model

↓

AI returns structured schedule recommendations

↓

App displays recommended shifts, study blocks, rest time, and warnings

↓

User chooses which suggestions to use

10. Example AI Input

```
{
  "calendar_events": [
    {
      "title": "Internship Interview",
      "date": "2026-06-17",
      "start_time": "14:00",
      "end_time": "14:30",
      "load_level": "high"
    },
    {
      "title": "Lunch with friend",
      "date": "2026-06-19",
```

```

    "start_time": "12:00",
    "end_time": "14:00",
    "load_level": "low"
  }
],
"user_preferences": {
  "minimum_sleep_hours": 7,
  "maximum_work_shifts_per_week": 2,
  "preferred_work_times": ["morning", "early afternoon"],
  "avoid_overbooking": true,
  "keep_one_evening_free": true
},
"weekly_priorities": {
  "main_priority": "internship preparation",
  "secondary_priority": "self-study",
  "social_life_goal": "at least one social plan",
  "current_energy_level": "medium",
  "current_stress_level": "high"
}
}

```

11. Example AI Output

```

{
  "weekly_summary": "This week should be planned carefully because the user has an internship interview and wants to prioritize preparation. Part-time work should be limited to two shifts.",
  "recommended_shift_availability": [
    {
      "day": "Monday",
      "date": "2026-06-15",
      "time": "10:00-15:00",
      "confidence": "high",
      "reason": "The user has no major events before or after this time, and the day still allows time for studying."
    },
    {
      "day": "Friday",
      "date": "2026-06-19",
      "time": "09:00-13:00",
      "confidence": "medium",
      "reason": "This is a good available block, but the user should keep the afternoon open for rest."
    }
  ]
}

```

```
}
],
"avoid_scheduling": [
  {
    "day": "Wednesday",
    "date": "2026-06-17",
    "reason": "The user has an internship interview, so adding work on this day may increase stress."
  }
],
"study_blocks": [
  {
    "day": "Tuesday",
    "time": "15:00-17:00",
    "focus": "interview preparation"
  },
  {
    "day": "Sunday",
    "time": "13:00-16:00",
    "focus": "self-study"
  }
],
"warnings": [
  "Avoid adding more than two work shifts this week.",
  "Keep at least one evening completely free."
]
}
```

12. Privacy and Safety Considerations

This app handles sensitive personal information, including calendar events, workload, mood, and stress levels. Therefore, privacy should be a major part of the design.

Important rules:

- Only send necessary information to the AI model.

- Do not send full private calendar details unless needed.

- Allow users to edit or delete stored personal data.

- Do not automatically create or change calendar events without user confirmation.

- Clearly explain that AI recommendations are suggestions, not final decisions.

A safer approach is to summarize sensitive information before sending it to the AI.

For example, instead of sending:

I cried because I was stressed about my internship interview.

The app could send:

Current stress level: high

Reason: upcoming interview

This keeps the app useful without exposing unnecessary personal details.

13. Development Phases

Phase 1: Basic UI

Build the main pages:

Login page

Dashboard

Weekly calendar view

User preferences page

Weekly priority input page

Phase 2: Google Calendar Integration

Implement Google login and fetch the user's calendar events for the selected week.

Phase 3: User Preferences

Allow the user to save basic scheduling rules, such as maximum work shifts, preferred work times, and minimum rest time.

Phase 4: AI Recommendation Engine

Send calendar data and user preferences to the AI model.

Return structured recommendations in JSON format.

Phase 5: Shift Availability Generator

Convert AI recommendations into a clean format that the user can copy and submit.

Phase 6: Portfolio Polish

Prepare the project for GitHub and portfolio use.

Add:

README

Screenshots

Demo video

Tech stack explanation

Problem statement

Future improvements

14. Possible Project Names

More formal names:

Adaptive Schedule Planner

Context-Aware Calendar Assistant

Personal Capacity Scheduler

AI Weekly Planner

Smart Availability Planner

Balanced Schedule Assistant

More memorable names:

Don't Overbook Me

SoftSchedule

Capacity Calendar

BalanceBoard

ShiftSense

LifeLoad

WeekWise

My favorite is still **Don't Overbook Me** because it clearly communicates the main value of the app. It sounds personal, useful, and slightly funny, which makes it memorable for a portfolio project.

15. Portfolio Value

This project would be strong for a portfolio because it shows several important skills:

- Full-stack web development
- API integration
- AI application design
- Prompt engineering
- Structured JSON output
- User-centered product thinking
- Privacy-aware design
- Real-world problem solving

It also has a clear personal story behind it: the app is built to solve the real problem of balancing internships, part-time work, studying, and personal life.

That makes it much more interesting than a generic to-do list app. It shows that the developer can identify a real user problem, design a practical solution, and use AI in a meaningful way.